ASSIGNMENT 1 (PAYAL, LEON, RENK) IFA_DS1

1. Perform an exploratory data analysis on the Cleveland heart disease dataset described above. You can reuse your code from the in-class project. Add at least two new plot types like pairplot and/or heatmap from the lecture (e.g. from the seaborn library in python).

In [1]:
```
import pandas as pd
import matplotlib.pyplot as plt
```

In [13]:
```
cleveland_patients = pd.read_csv('/Users/payalpriyadarshini/Downloads/heart+disease/processed.cleveland.data',
cleveland_patients.dropna() #drops rows with missing values
cleveland_patients.head()
```

Out[13]:

	63.0	1.0	1.0.1	145.0	233.0	1.0.2	2.0	150.0	0.0	2.3	3.0	0.0.1	6.0	0
0	67.0	1.0	4.0	160.0	286.0	0.0	2.0	108.0	1.0	1.5	2.0	3.0	3.0	2
1	67.0	1.0	4.0	120.0	229.0	0.0	2.0	129.0	1.0	2.6	2.0	2.0	7.0	1
2	37.0	1.0	3.0	130.0	250.0	0.0	0.0	187.0	0.0	3.5	3.0	0.0	3.0	0
3	41.0	0.0	2.0	130.0	204.0	0.0	2.0	172.0	0.0	1.4	1.0	0.0	3.0	0
4	56.0	1.0	2.0	120.0	236.0	0.0	0.0	178.0	0.0	0.8	1.0	0.0	3.0	0

In [14]:
```
cleveland_patients = pd.read_csv(
    '/Users/payalpriyadarshini/Downloads/heart+disease/processed.cleveland.data',
    na_values='?',  # Replace '?' with NaN
    header=None  # Do not assume the first row is a header
)
# Display the first few rows to verify
cleveland_patients.head()
```

Out[14]:

	0	1	2	3	4	5	6	7	8	9	10	11	12	13
0	63.0	1.0	1.0	145.0	233.0	1.0	2.0	150.0	0.0	2.3	3.0	0.0	6.0	0
1	67.0	1.0	4.0	160.0	286.0	0.0	2.0	108.0	1.0	1.5	2.0	3.0	3.0	2
2	67.0	1.0	4.0	120.0	229.0	0.0	2.0	129.0	1.0	2.6	2.0	2.0	7.0	1
3	37.0	1.0	3.0	130.0	250.0	0.0	0.0	187.0	0.0	3.5	3.0	0.0	3.0	0
4	41.0	0.0	2.0	130.0	204.0	0.0	2.0	172.0	0.0	1.4	1.0	0.0	3.0	0

In [18]:
```
cleveland_patients.columns = ['age', 'sex', 'cp', 'trestbps', 'chol', 'fbs', 'restecg', 'thalach',
                              'exang', 'oldpeak', 'slope', 'ca', 'thal', 'goal']
cleveland_patients.head()
```

Out[18]:

	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	slope	ca	thal	goal
0	63.0	1.0	1.0	145.0	233.0	1.0	2.0	150.0	0.0	2.3	3.0	0.0	6.0	0
1	67.0	1.0	4.0	160.0	286.0	0.0	2.0	108.0	1.0	1.5	2.0	3.0	3.0	2
2	67.0	1.0	4.0	120.0	229.0	0.0	2.0	129.0	1.0	2.6	2.0	2.0	7.0	1
3	37.0	1.0	3.0	130.0	250.0	0.0	0.0	187.0	0.0	3.5	3.0	0.0	3.0	0
4	41.0	0.0	2.0	130.0	204.0	0.0	2.0	172.0	0.0	1.4	1.0	0.0	3.0	0

In [20]:
```
print(cleveland_patients.shape)
```

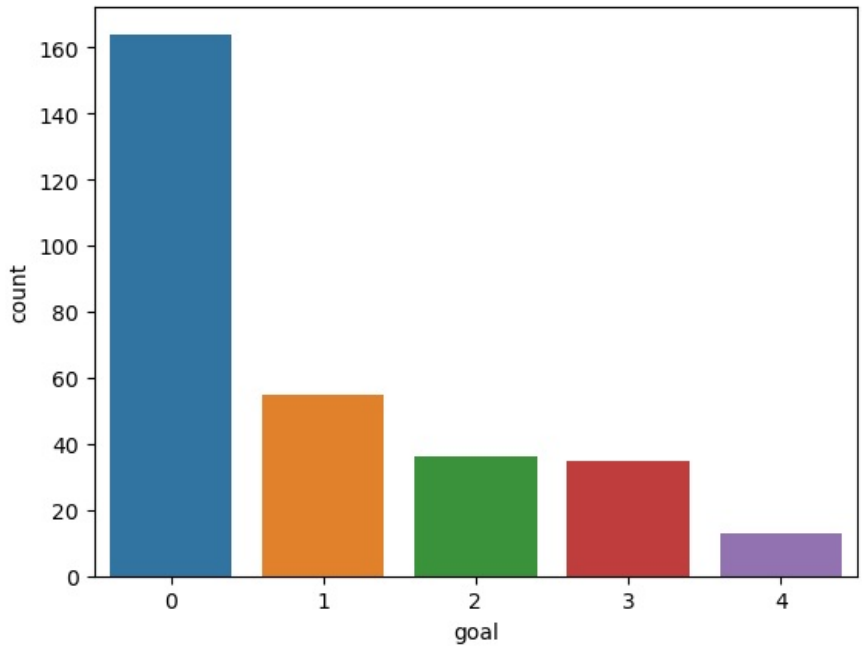
```
(303, 14)
```

In [22]:
```
print(cleveland_patients.groupby('goal').size()) # to check the variation of presence of a heart disease in a p
```

```
goal
0    164
1     55
2     36
3     35
4     13
dtype: int64
```

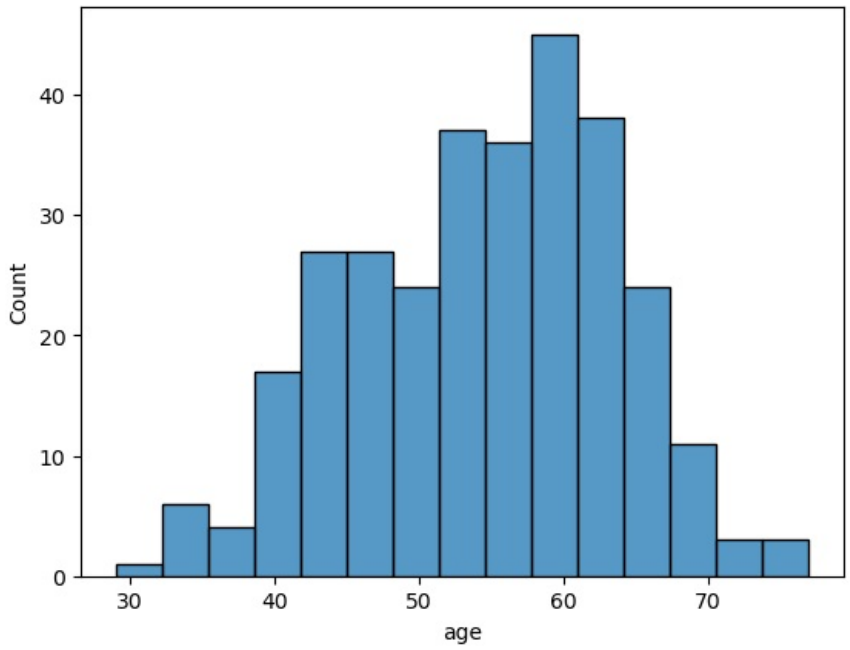
In [21]:
```
import seaborn as sns
```

In [31]:
```
sns.countplot(x=cleveland_patients['goal'],label="Count") #as directed in the assignment shows patients with pr
plt.show()
```



In [32]:
```
sns.histplot(data=cleveland_patients, x='age', bins=15) #another plot under seaborn library showing the variati
plt.show()
```

```
/opt/anaconda3/lib/python3.11/site-packages/seaborn/_oldcore.py:1119: FutureWarning: use_inf_as_na option is de
precated and will be removed in a future version. Convert inf values to NaN before operating instead.
  with pd.option_context('mode.use_inf_as_na', True):
```



In [48]:
```
#set up the 2x2 grid for subplots
fig, axes = plt.subplots(2, 2, figsize=(12, 10))
fig.suptitle('Boxplots of Key Health Parameters')

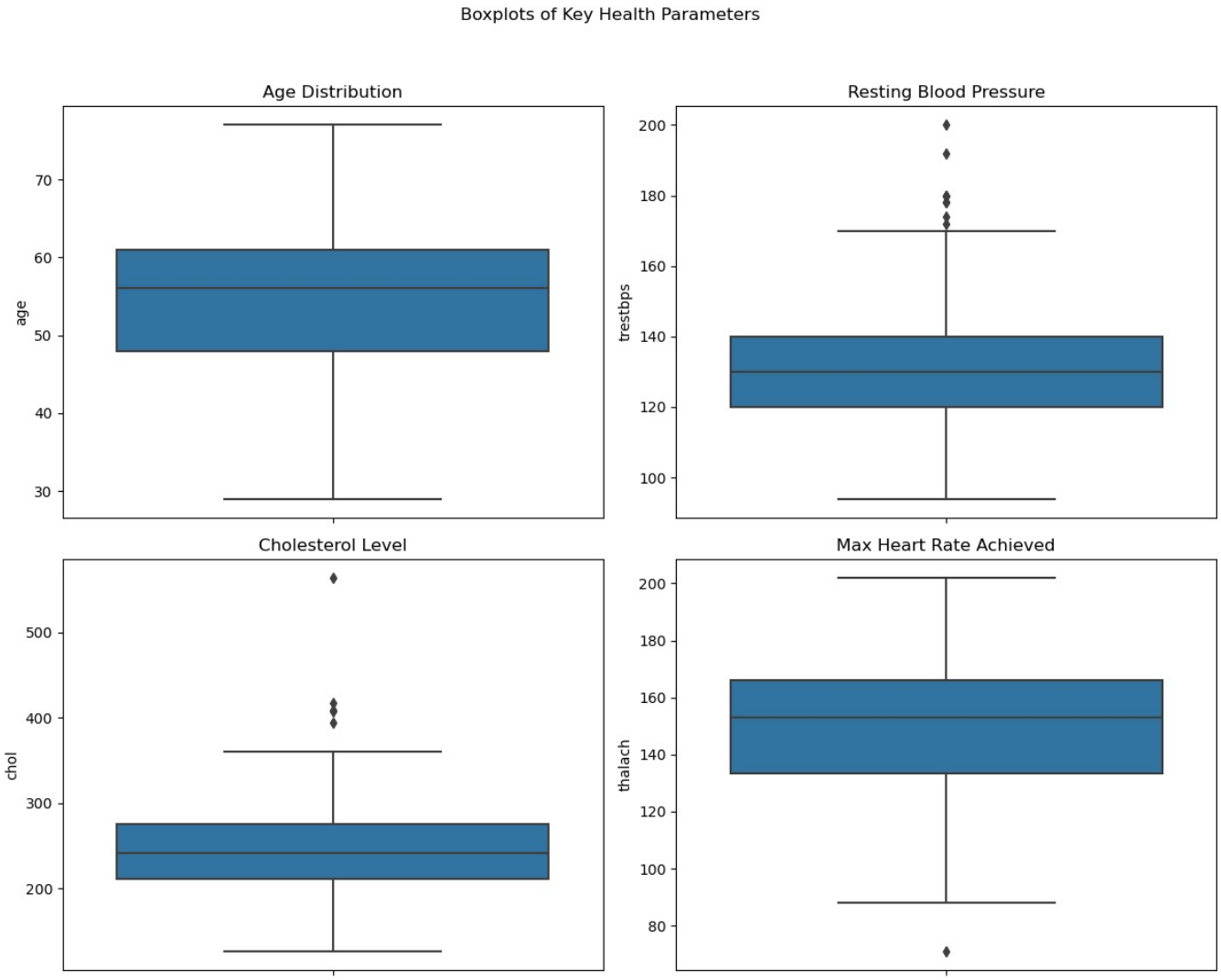
sns.boxplot(data=cleveland_patients, y='age', ax=axes[0, 0])
axes[0, 0].set_title('Age Distribution')

sns.boxplot(data=cleveland_patients, y='trestbps', ax=axes[0, 1])
axes[0, 1].set_title('Resting Blood Pressure')

sns.boxplot(data=cleveland_patients, y='chol', ax=axes[1, 0])
axes[1, 0].set_title('Cholesterol Level')

sns.boxplot(data=cleveland_patients, y='thalach', ax=axes[1, 1])
axes[1, 1].set_title('Max Heart Rate Achieved')

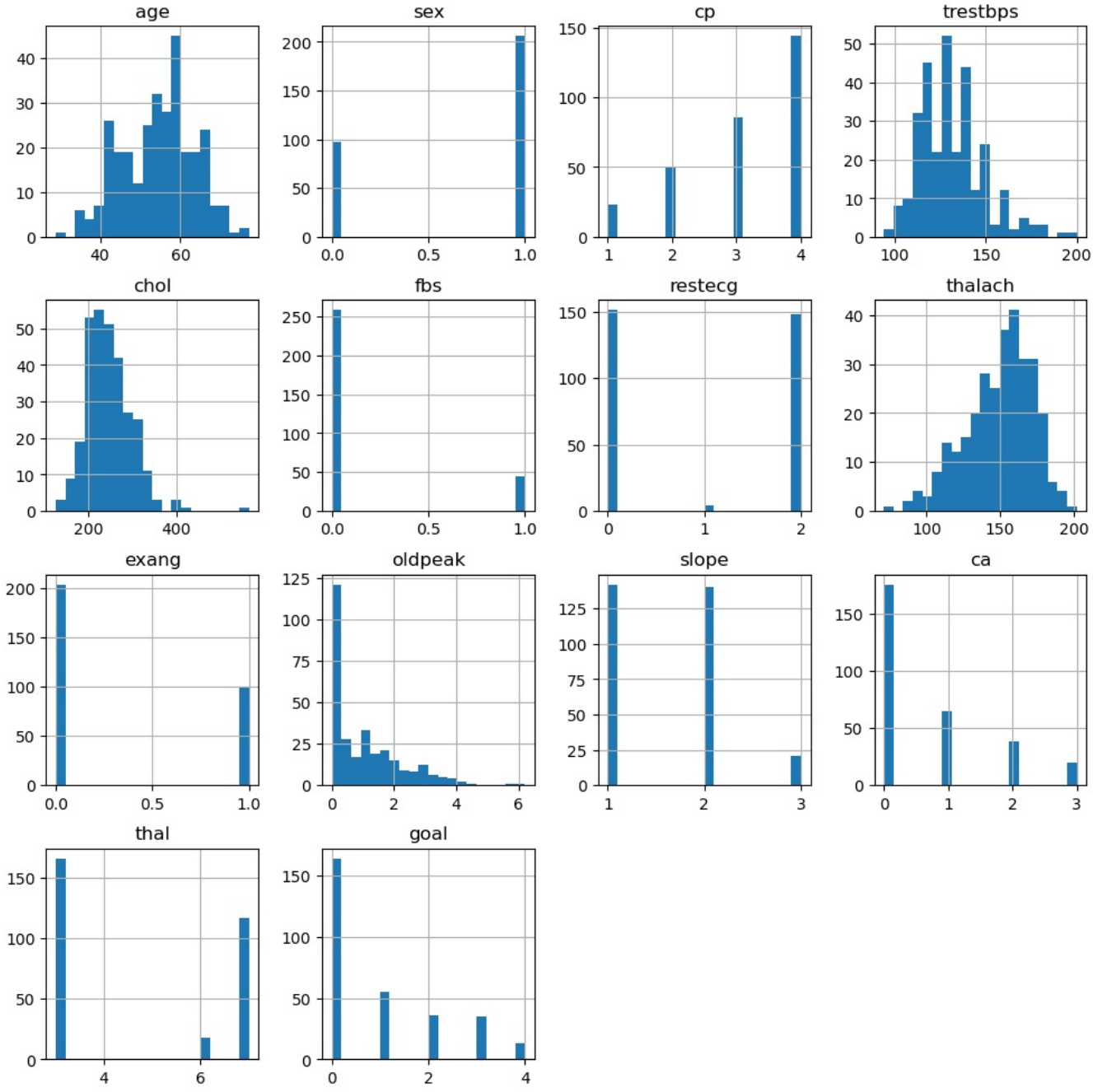
plt.tight_layout(rect=[0, 0.03, 1, 0.95])
plt.show()
```



In [49]:
```
import pylab as pl
```

In [50]:
```
cleveland_patients.hist(bins=20, figsize=(12,12))
plt.suptitle("Histogram for each numeric input variable")
plt.savefig('patients_hist')
plt.show()
```

Histogram for each numeric input variable

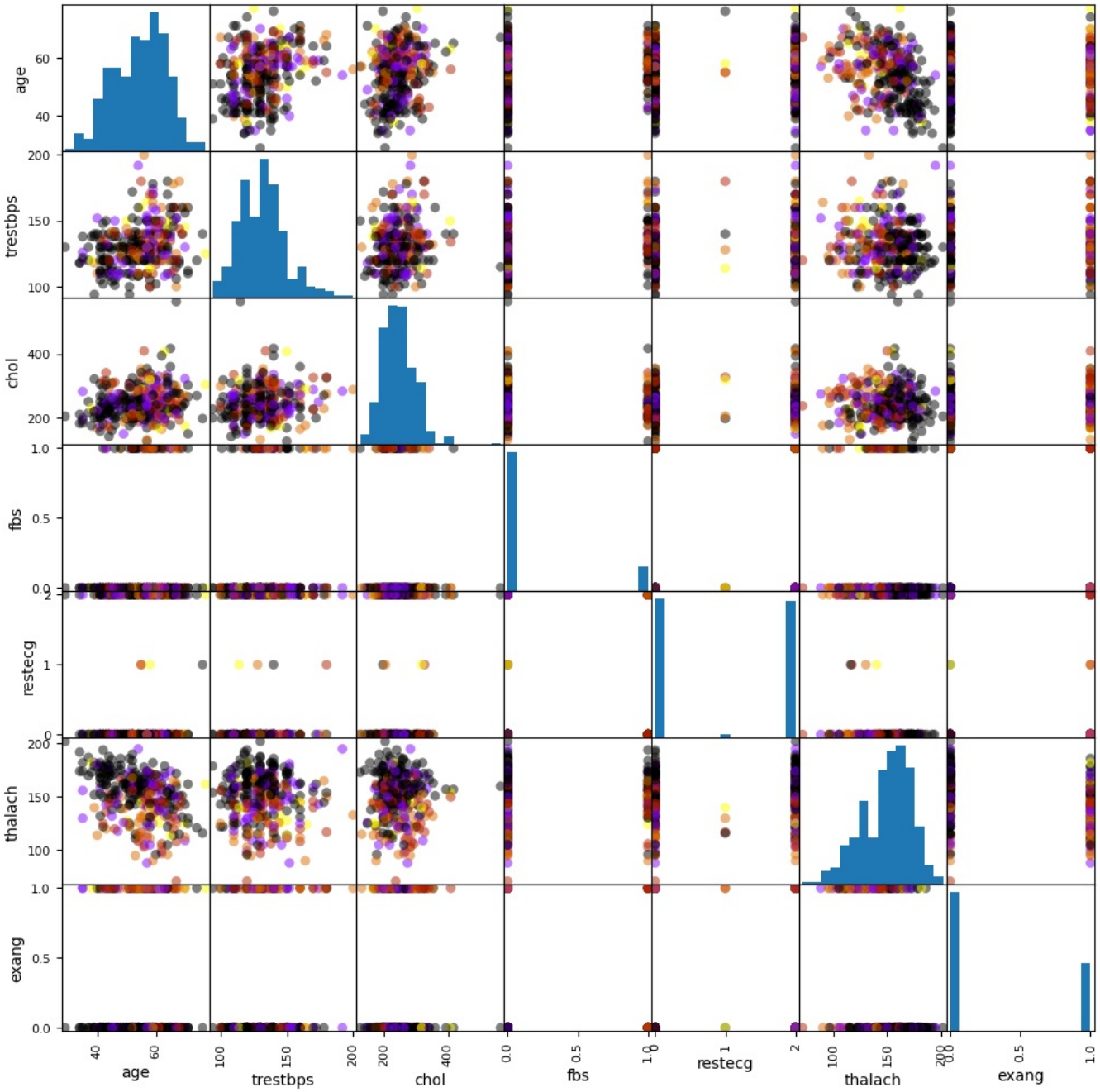


In [51]:
```
from pandas.plotting import scatter_matrix
from matplotlib import cm
```

In [52]:
```
#scatter matrix
feature_names = ['age', 'trestbps', 'chol', 'fbs', 'restecg', 'thalach', 'exang']
X = cleveland_patients[feature_names].copy()
y = cleveland_patients['goal'].copy()
cmap = cm.get_cmap('gnuplot')
scatter = scatter_matrix(X, c = y, marker = 'o', s=40, hist_kwds={'bins':15}, figsize=(12,12), cmap = cmap)
plt.suptitle('Scatter-matrix for important input variables')
plt.savefig('fruits_scatter_matrix')
```

```
/var/folders/6l/tc6k8srj35x2szlp4rvgtzd00000gn/T/ipykernel_70433/1736382961.py:5: MatplotlibDeprecationWarning:
The get_cmap function was deprecated in Matplotlib 3.7 and will be removed two minor releases later. Use ``matp
lotlib.colormaps[name]`` or ``matplotlib.colormaps.get_cmap(obj)`` instead.
  cmap = cm.get_cmap('gnuplot')
```

Scatter-matrix for important input variables

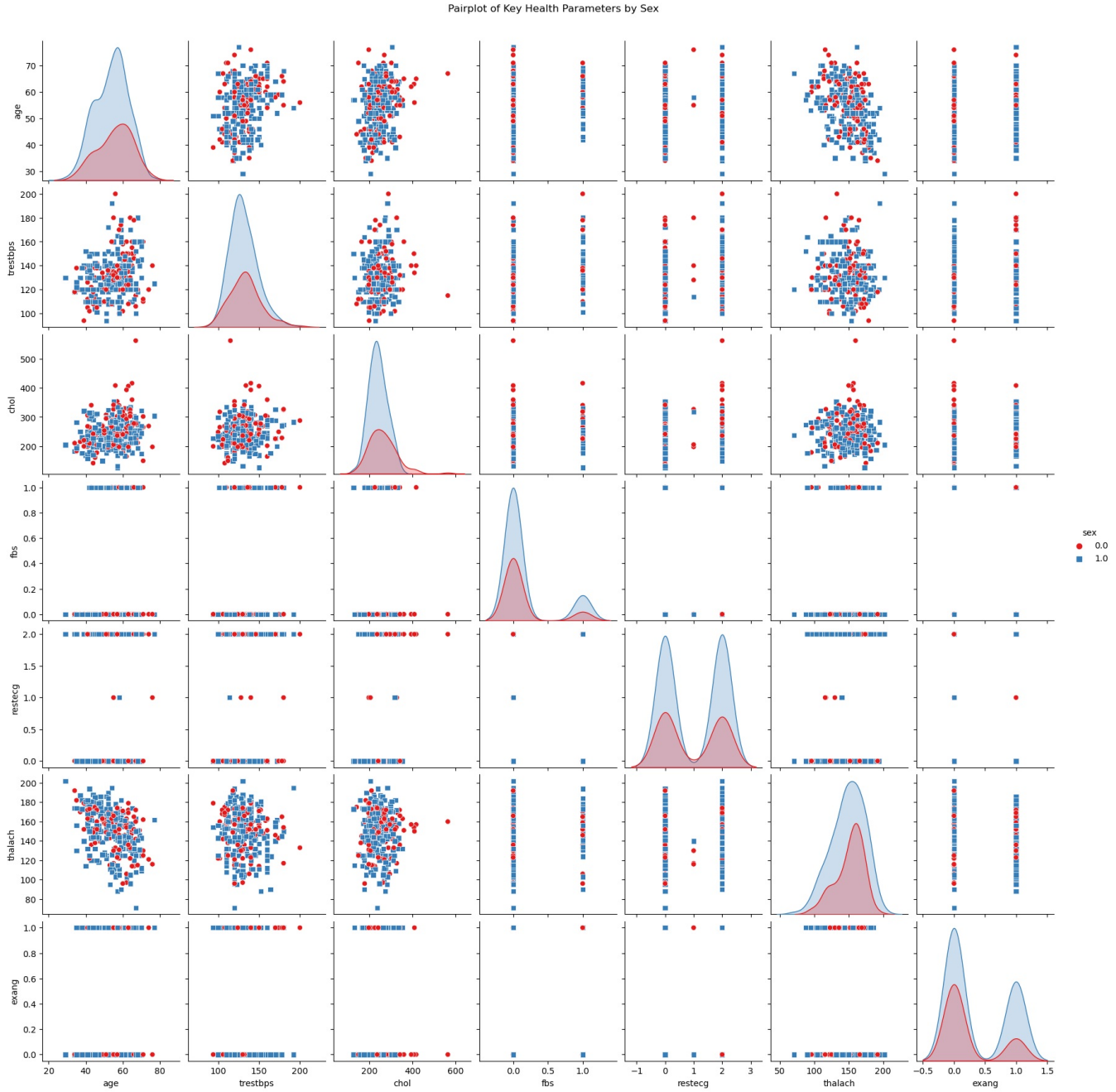


In [53]:

```
X.loc[:, 'sex'] = cleveland_patients['sex']

sns.pairplot(X, hue='sex', palette='Set1', diag_kind='kde', markers=["o", "s"])
plt.suptitle('Pairplot of Key Health Parameters by Sex', y=1.02)
plt.show()
```

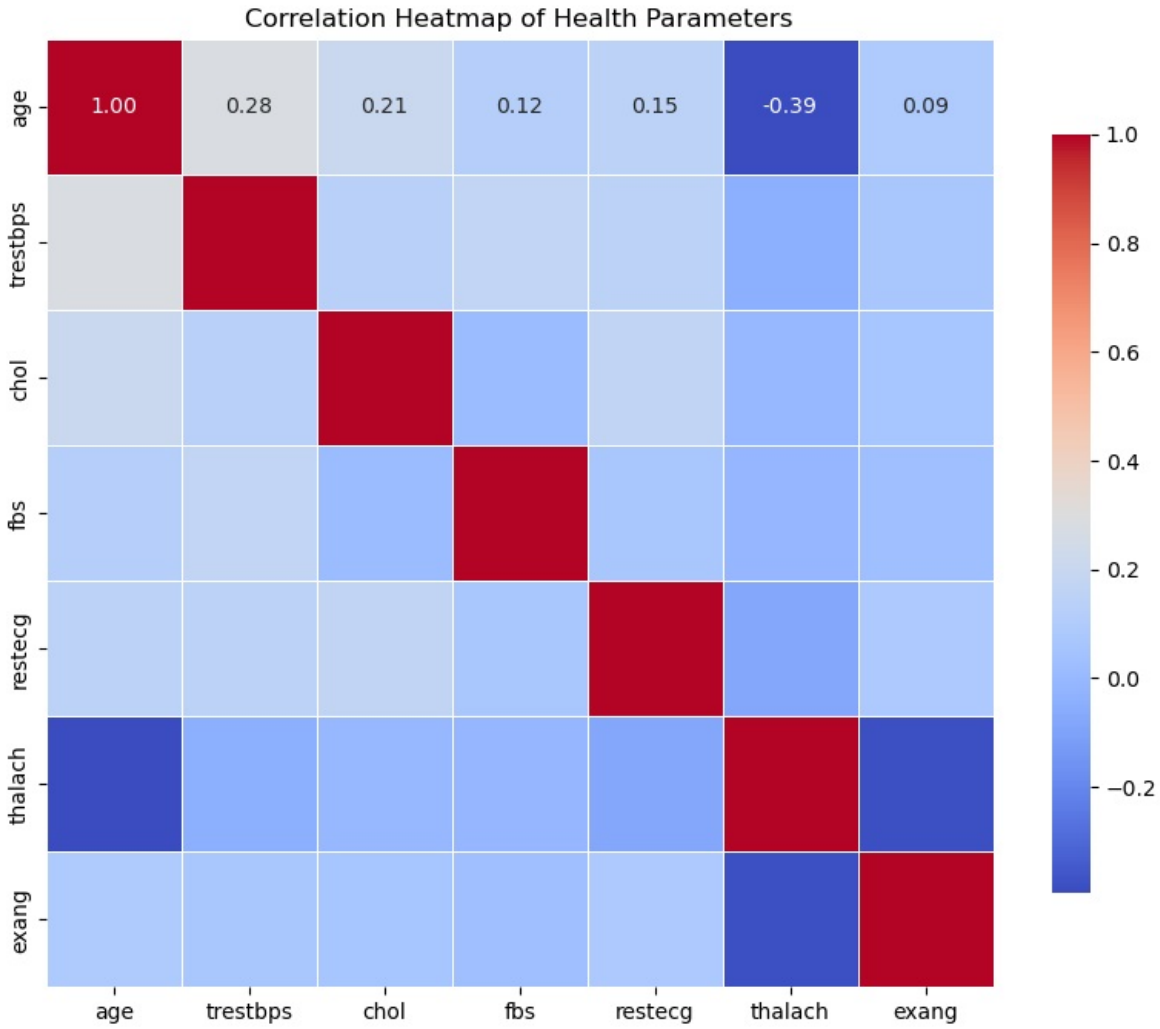
```
/opt/anaconda3/lib/python3.11/site-packages/seaborn/_oldcore.py:1119: FutureWarning: use_inf_as_na option is deprecated and will be removed in a future version. Convert inf values to NaN before operating instead.
  with pd.option_context('mode.use_inf_as_na', True):
/opt/anaconda3/lib/python3.11/site-packages/seaborn/_oldcore.py:1119: FutureWarning: use_inf_as_na option is deprecated and will be removed in a future version. Convert inf values to NaN before operating instead.
  with pd.option_context('mode.use_inf_as_na', True):
/opt/anaconda3/lib/python3.11/site-packages/seaborn/_oldcore.py:1119: FutureWarning: use_inf_as_na option is deprecated and will be removed in a future version. Convert inf values to NaN before operating instead.
  with pd.option_context('mode.use_inf_as_na', True):
/opt/anaconda3/lib/python3.11/site-packages/seaborn/_oldcore.py:1119: FutureWarning: use_inf_as_na option is deprecated and will be removed in a future version. Convert inf values to NaN before operating instead.
  with pd.option_context('mode.use_inf_as_na', True):
/opt/anaconda3/lib/python3.11/site-packages/seaborn/_oldcore.py:1119: FutureWarning: use_inf_as_na option is deprecated and will be removed in a future version. Convert inf values to NaN before operating instead.
  with pd.option_context('mode.use_inf_as_na', True):
/opt/anaconda3/lib/python3.11/site-packages/seaborn/_oldcore.py:1119: FutureWarning: use_inf_as_na option is deprecated and will be removed in a future version. Convert inf values to NaN before operating instead.
  with pd.option_context('mode.use_inf_as_na', True):
/opt/anaconda3/lib/python3.11/site-packages/seaborn/_oldcore.py:1119: FutureWarning: use_inf_as_na option is deprecated and will be removed in a future version. Convert inf values to NaN before operating instead.
  with pd.option_context('mode.use_inf_as_na', True):
```



In [55]:

```
X = cleveland_patients[feature_names].copy()
y = cleveland_patients['goal'].copy()
correlation_matrix = X.corr()

plt.figure(figsize=(10, 8))
sns.heatmap(correlation_matrix, annot=True, fmt=".2f", cmap='coolwarm', square=True,
            cbar_kws={"shrink": .8}, linewidths=.5)
plt.title('Correlation Heatmap of Health Parameters')
plt.show()
```



CLASSIFIERS-

In [56]:
```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import MinMaxScaler
```

In [68]:
```
patients = cleveland_patients.apply(pd.to_numeric) #converting to numeric removes errors
patients.fillna(patients.mean(), inplace=True) # to remove Nan values and fill them with mean of nearest values
X = patients.copy()
X = X.drop(columns=['goal'])

X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=0)
scaler = MinMaxScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

In [69]:
```
from sklearn.metrics import accuracy_score, recall_score, confusion_matrix
import numpy as np
```

In [70]:
```
def specificity(y_true, y_pred):
    cm = confusion_matrix(y_true, y_pred)
    specificities = []
    for i in range(cm.shape[0]):  # Loop through each class
        tn = np.sum(cm) - np.sum(cm[i, :]) - np.sum(cm[:, i]) + cm[i, i]  # True Negatives
        fp = np.sum(cm[:, i]) - cm[i, i]  # False Positives
        specificities.append(tn / (tn + fp) if (tn + fp) > 0 else 0)
    return np.mean(specificities)  # Return average specificity
```

In [71]:
```
results = {
    'Model': [],
    'Accuracy': [],
    'Sensitivity': [],
    'Specificity': []
}
```

In [72]:
```
#logistic Regression
from sklearn.linear_model import LogisticRegression

X = patients.drop(columns='goal')
y = patients['goal'].apply(lambda x: 1 if x > 0 else 0)  # Convert to BINARY classification


logreg = LogisticRegression()
```

```
logreg.fit(X_train, y_train)
print('Accuracy of Logistic regression classifier on training set: {:.2f}'
     .format(logreg.score(X_train, y_train)))
print('Accuracy of Logistic regression classifier on test set: {:.2f}'
     .format(logreg.score(X_test, y_test)))

y_pred = logreg.predict(X_test)

train_accuracy = logreg.score(X_train, y_train)
test_accuracy = logreg.score(X_test, y_test)
sensitivity = recall_score(y_test, y_pred, average='macro')
speci = specificity(y_test, y_pred)

results['Model'].append('logistic regression')
results['Accuracy'].append(test_accuracy)
results['Sensitivity'].append(sensitivity)
results['Specificity'].append(speci)
```

```
Accuracy of Logistic regression classifier on training set: 0.65
Accuracy of Logistic regression classifier on test set: 0.59
```

In [73]:

```
#KNN
from sklearn.neighbors import KNeighborsClassifier
knn = KNeighborsClassifier()
knn.fit(X_train, y_train)
print('Accuracy of K-NN classifier on training set: {:.2f}'
     .format(knn.score(X_train, y_train)))
print('Accuracy of K-NN classifier on test set: {:.2f}'
     .format(knn.score(X_test, y_test)))

y_pred = knn.predict(X_test)

train_accuracy = knn.score(X_train, y_train)
test_accuracy = knn.score(X_test, y_test)
sensitivity = recall_score(y_test, y_pred, average='macro')
speci = specificity(y_test, y_pred)

results['Model'].append('knn')
results['Accuracy'].append(test_accuracy)
results['Sensitivity'].append(sensitivity)
results['Specificity'].append(speci)
```

```
Accuracy of K-NN classifier on training set: 0.67
Accuracy of K-NN classifier on test set: 0.55
```

In [74]:

```
#Gaussian Naive Bayes
from sklearn.naive_bayes import GaussianNB
gnb = GaussianNB()
gnb.fit(X_train, y_train)
print('Accuracy of GNB classifier on training set: {:.2f}'
     .format(gnb.score(X_train, y_train)))
print('Accuracy of GNB classifier on test set: {:.2f}'
     .format(gnb.score(X_test, y_test)))

train_accuracy = gnb.score(X_train, y_train)
test_accuracy = gnb.score(X_test, y_test)
sensitivity = recall_score(y_test, y_pred, average='macro')
speci = specificity(y_test, y_pred)

results['Model'].append('gaussian naive bayes')
results['Accuracy'].append(test_accuracy)
results['Sensitivity'].append(sensitivity)
results['Specificity'].append(speci)
```

```
Accuracy of GNB classifier on training set: 0.41
Accuracy of GNB classifier on test set: 0.39
```

In [75]:

```
#Support Vector Machine
from sklearn.svm import SVC
svm = SVC()
svm.fit(X_train, y_train)
print('Accuracy of SVM classifier on training set: {:.2f}'
     .format(svm.score(X_train, y_train)))
print('Accuracy of SVM classifier on test set: {:.2f}'
     .format(svm.score(X_test, y_test)))

train_accuracy = svm.score(X_train, y_train)
test_accuracy = svm.score(X_test, y_test)
sensitivity = recall_score(y_test, y_pred, average='macro')
speci = specificity(y_test, y_pred)

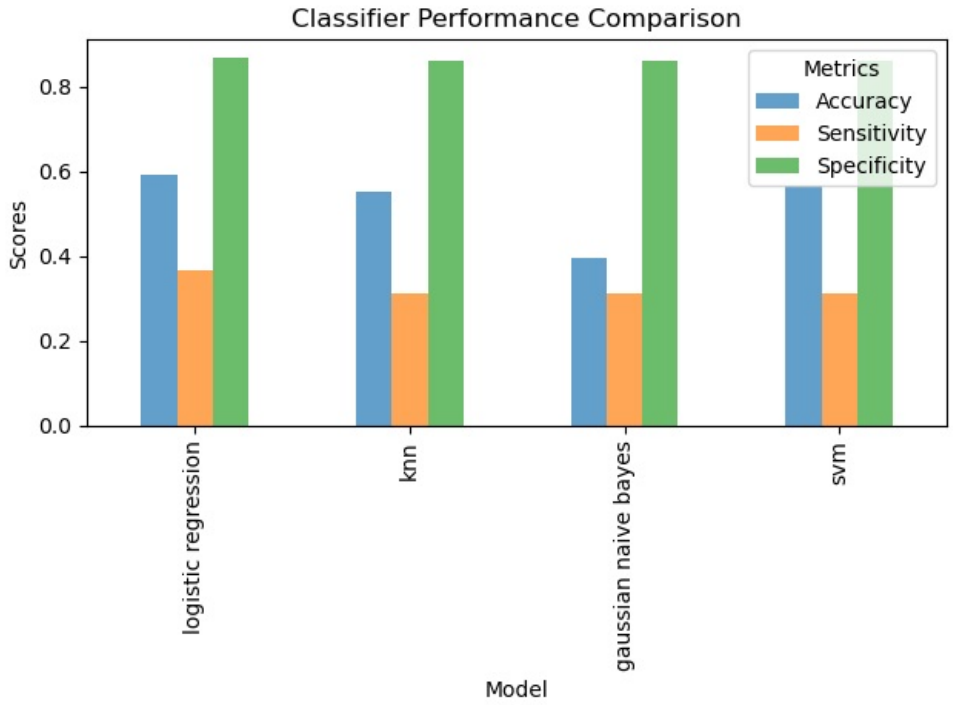
results['Model'].append('svm')
results['Accuracy'].append(test_accuracy)
results['Sensitivity'].append(sensitivity)
results['Specificity'].append(speci)
```

```
Accuracy of SVM classifier on training set: 0.70
Accuracy of SVM classifier on test set: 0.57
```

Grouped bar chart with regard to accuracy, sensitivity and specificity

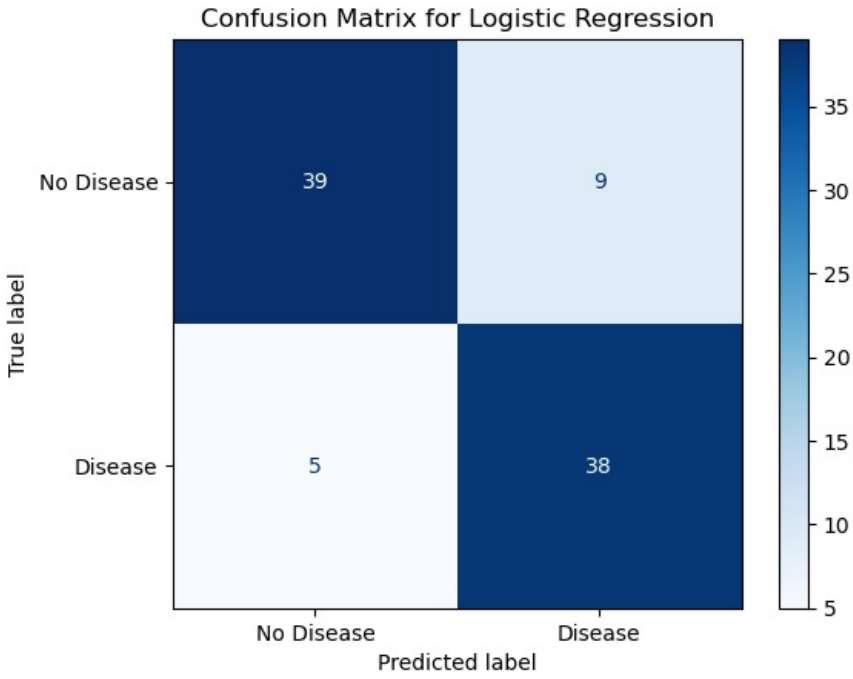
In [76]:
```
results_df = pd.DataFrame(results)
```

In [77]:
```
# Create grouped bar chart
results_df.set_index('Model').plot(kind='bar', alpha=0.7)
plt.title('Classifier Performance Comparison')
plt.ylabel('Scores')
plt.legend(title='Metrics')
plt.tight_layout()
plt.show()
```



In [38]:
```
import pandas as pd
from sklearn.model_selection import train_test_split  # Import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import confusion_matrix
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay #important
import seaborn as sns
import matplotlib.pyplot as plt
import numpy
```

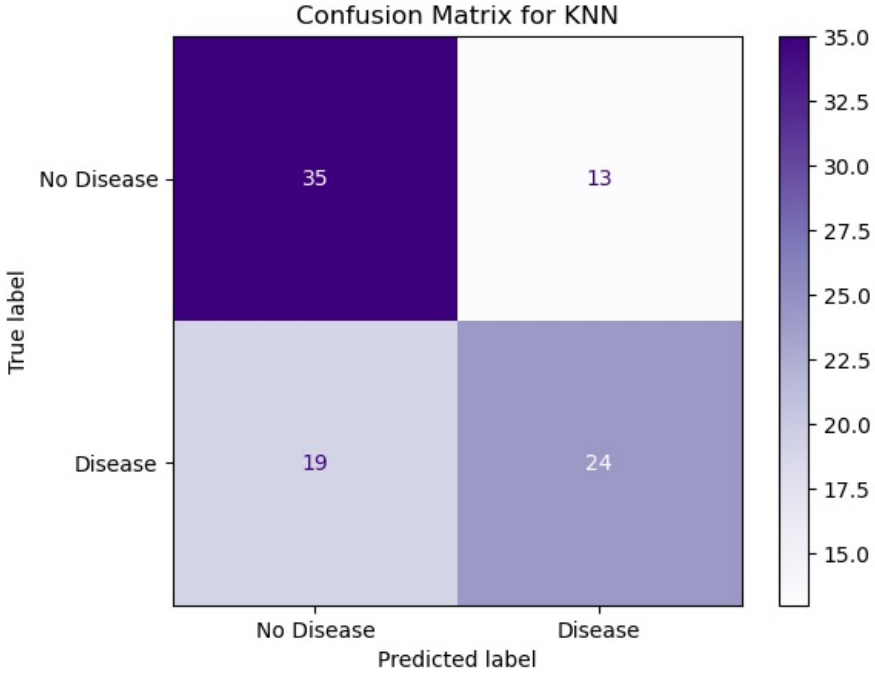
In [41]:
```
#logistic regression
from sklearn import metrics
patients = cleveland_patients.apply(pd.to_numeric) #convert to numeric to ensure there are no errors
patients.fillna(patients.mean(), inplace=True) # to remove Nan values and fill them with mean of nearest values
X = patients.drop(columns='goal')
y = patients['goal'].apply(lambda x: 1 if x > 0 else 0)  # Convert to BINARY classification
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42) ## Split the data int
model = LogisticRegression(max_iter=1000) # Created and trained the logistic regression model
model.fit(X_train, y_train)
y_pred = model.predict(X_test) # to make predictions
cm = confusion_matrix(y_test, y_pred) #confusion matrix
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=['No Disease', 'Disease'])
disp.plot(cmap='Blues') # Plot
plt.title('Confusion Matrix for Logistic Regression')
plt.show()
```



KNN Confusion matrix and heatmap

In [85]:
```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay

patients = cleveland_patients.apply(pd.to_numeric) # to remove Nan values and fill them with mean of nearest va
patients.fillna(patients.mean(), inplace=True)
X = patients.drop(columns='goal')
y = patients['goal'].apply(lambda x: 1 if x > 0 else 0)  # Convert to binary classification
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
knn_model = KNeighborsClassifier(n_neighbors=10)  # Set n_neighbors to 10 in KNN model
knn_model.fit(X_train, y_train)
y_pred_knn = knn_model.predict(X_test)
cm_knn = confusion_matrix(y_test, y_pred_knn)
disp_knn = ConfusionMatrixDisplay(confusion_matrix=cm_knn, display_labels=['No Disease', 'Disease'])
disp_knn.plot(cmap='Purples')
plt.title('Confusion Matrix for KNN')
plt.show()
```



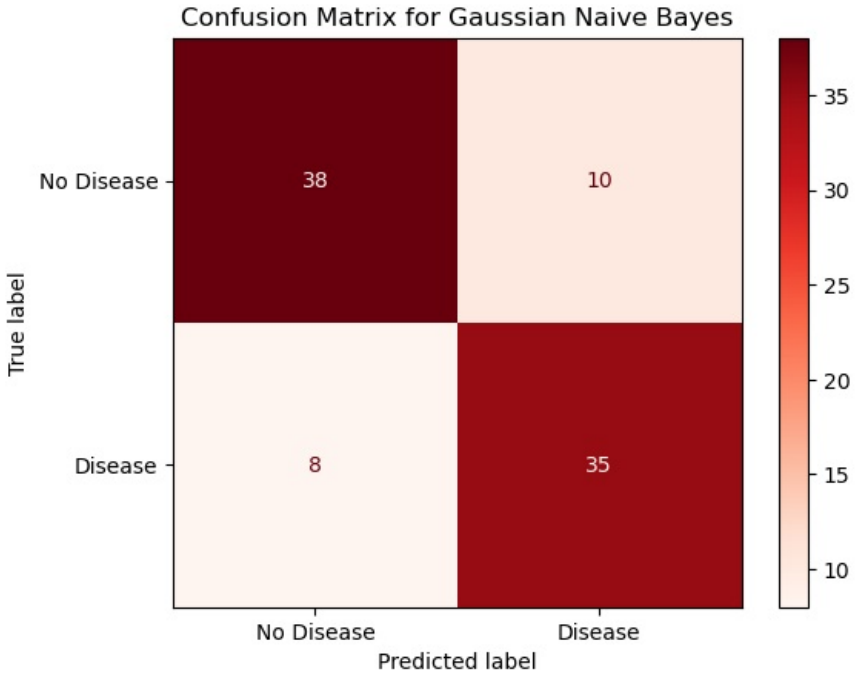
In [86]:
```
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay

patients = cleveland_patients.apply(pd.to_numeric)
patients.fillna(patients.mean(), inplace=True) #removes NaN values

# Convert to binary classification

X = patients.drop(columns='goal') # represents feature matrix
y = patients['goal'].apply(lambda x: 1 if x > 0 else 0)  #to show presence and absence of disease ie. target va
```

```
# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
gnb_model = GaussianNB()  #Gaussian Naive Bayes model
gnb_model.fit(X_train, y_train)
y_pred_gnb = gnb_model.predict(X_test)
cm_gnb = confusion_matrix(y_test, y_pred_gnb)
disp_gnb = ConfusionMatrixDisplay(confusion_matrix=cm_gnb, display_labels=['No Disease', 'Disease'])
disp_gnb.plot(cmap='Reds')
plt.title('Confusion Matrix for Gaussian Naive Bayes')
plt.show()
```

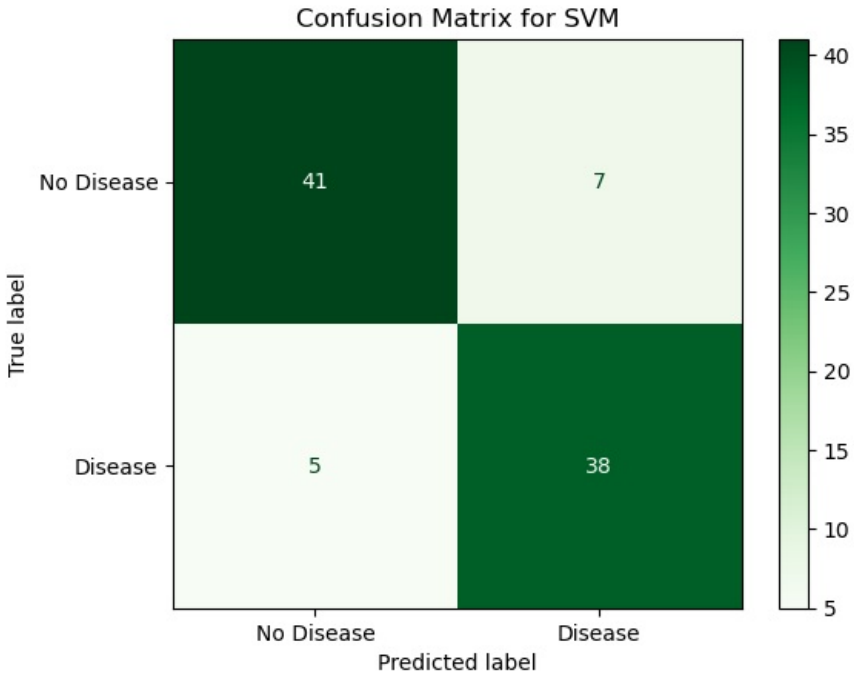


SVM matrix and heatmap

In [47]:
```
from sklearn.svm import SVC
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
patients = cleveland_patients.apply(pd.to_numeric)
patients.fillna(patients.mean(), inplace=True)

X = patients.drop(columns='goal')
y = patients['goal'].apply(lambda x: 1 if x > 0 else 0)  # Convert to binary classification
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

# Create and train the SVM model
svm_model = SVC(kernel='linear') #??
svm_model.fit(X_train, y_train)
y_pred_svm = svm_model.predict(X_test)
cm_svm = confusion_matrix(y_test, y_pred_svm)
disp_svm = ConfusionMatrixDisplay(confusion_matrix=cm_svm, display_labels=['No Disease', 'Disease'])
disp_svm.plot(cmap='Greens')
plt.title('Confusion Matrix for SVM')
plt.show()
```



1. Create and plot ROC curves for your classifiers (2-class problem only).

In [ ]:

```
from sklearn.metrics import roc_curve, roc_auc_score
from sklearn.preprocessing import label_binarize
from sklearn.multiclass import OneVsRestClassifier
import matplotlib.pyplot as plt
n_classes = len(set(y))  # Set number of classes
y_test_bin = label_binarize(y_test, classes=range(n_classes))
classifiers = {
    'Logistic Regression': logreg,
    'K-Nearest Neighbors': knn,
    'Gaussian Naive Bayes': gnb,
    'Support Vector Machine': svm
}

# Plot setup
plt.figure(figsize=(8, 6))

# Loop through classifiers
for name, clf in classifiers.items():
    # Get probability scores if available, else skip
    if hasattr(clf, "predict_proba"):
        y_proba = clf.predict_proba(X_test)[:, 1]
    else:
        y_proba = clf.decision_function(X_test)

    # Compute ROC curve and AUC score
    fpr, tpr, _ = roc_curve(y_test, y_proba)
    auc_score = roc_auc_score(y_test, y_proba)

    # Plot ROC curve
    plt.plot(fpr, tpr, label=f'{name} (AUC = {auc_score:.2f})')

# Plot the random classifier line
plt.plot([0, 1], [0, 1], 'k--')

# Customize plot
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC Curves')
plt.legend(loc='lower right')
plt.show()
```

Loading [MathJax]/jax/output/CommonHTML/fonts/TeX/fontdata.js